

SQL Learning Log

Notes on SQL Fundamentals, Joins, Aggregation, and Table Operations

Chandrachud Sahi

22 June 2026 - 25 June 2026

DATE: 22/06/2026

SQL FUNDAMENTALS

SQL - Structured Query Language is a language built for managing, reshaping, and pulling data out of a relational database. It does not distinguish between upper and lower case.

Relational Database - A relational database is a group of connected two-dimensional tables. Each table holds attributes (its columns) and data (its rows), where a row is one instance of those attributes.

Query - A query is a statement that describes what data is wanted, where to look for it, and how it should be reshaped before it comes back. Queries follow a defined syntax.

SELECT - The SELECT statement pulls the data that is needed from a table.

```
SELECT column1, column2 FROM table_name; (Selects specific columns)
```

```
SELECT * FROM table_name; (Selects every column)
```

WHERE - The WHERE clause pulls data from a table based on whether each row meets a given condition. It is checked against every row under the column in question.

NOTE - Filtering data down early means fewer unnecessary rows are returned, which makes a query run faster.

```
SELECT column FROM table_name WHERE column = condition; (Returns only the rows that meet the condition)
```

OPERATORS (for numbers)

Operator	Meaning	Example
=, !=, <, >, >=, <=	Standard numerical comparisons	Id != 9
BETWEEN	Number falls within a range of two values (inclusive)	Score BETWEEN 40 AND 60
NOT BETWEEN	Number falls outside a range of two values (inclusive)	Score NOT BETWEEN 45 AND 55
IN (.....)	Number exists in a given list	Score IN (40, 45, 50)
NOT IN (.....)	Number does not exist in a given list	Score NOT IN (41, 43)

OPERATORS (for strings)

Operator	Meaning	Example
=	Case-sensitive exact match	column = 'String'
!= or <>	Case-sensitive exact mismatch	column != 'String'
LIKE	Case-insensitive exact match	column LIKE 'String'
NOT LIKE	Case-insensitive exact mismatch	column NOT LIKE 'String'
%	Wildcard matching zero or more characters (only with LIKE / NOT LIKE)	column LIKE '%String%'
_	Wildcard matching exactly one character (only with LIKE / NOT LIKE)	column LIKE 'AB_'

Operator	Meaning	Example
IN (.....)	String exists in a given list	column IN ('String1','String2')
NOT IN (.....)	String does not exist in a given list	column NOT IN ('String1','String2')

NOTE - (i) Strings always need to be quoted, otherwise the parser cannot tell a word in the string apart from a SQL keyword.

NOTE - (ii) These operators work well enough for most cases, but true full-text search is better handled by a dedicated engine such as Apache Lucene or Sphinx, which are purpose-built for it and support features like internationalization that plain SQL operators do not.

NOTE - (iii) % and _ are called wildcards, since — much like a joker in a card deck can stand in for any card — they can stand in for any sequence of characters without a fixed value of their own.

NOTE - (iv) LIKE used without any wildcards behaves just like a case-insensitive =.

DISTINCT - This keyword throws away rows that repeat a column value. It goes right after SELECT.

```
SELECT DISTINCT column FROM table_name WHERE conditions;
```

NOTE - DISTINCT removes duplicate rows without exception.

ORDER BY - This keyword sorts the results in ascending or descending order.

```
SELECT column FROM table_name WHERE conditions ORDER BY ASC/DESC;
```

NOTE - Once an ORDER BY clause is given, every row is sorted alpha-numerically by that column's value — text columns go A to Z and number columns go smallest to largest by default. When a column mixes letters and digits as text, sorting happens character by character left to right, which can produce results that look out of order.

LIMIT - Used together with ORDER BY, this clause caps how many rows the results contain.

```
SELECT column FROM table_name WHERE conditions ORDER BY column ASC LIMIT number;
```

OFFSET - Used together with ORDER BY, this clause sets where counting the returned rows should begin.

```
SELECT column FROM table_name WHERE conditions ORDER BY column ASC LIMIT number OFFSET number;
```

NOTE - LIMIT and OFFSET are fine when the exact rows needed are known in advance, but they get impractical on large tables where row positions shift or are unknown. Window functions such as ROW_NUMBER() and RANK() solve this by assigning a rank to every row regardless of table size, so filtering by position no longer needs a manually calculated offset.

BOOKS

S.NO.	TITLE	AUTHOR	YEAR
1	DUNE	HERBERT	1965
2	EMMA	AUSTEN	1815
3	IT	KING	1986
4	CIRCE	MILLER	2018

1. `SELECT * FROM books;` (Gives the entire table)
2. `SELECT title, year FROM books;` (Gives every book title with its year)
3. `SELECT title FROM books WHERE year = 1965;` (Gives the book title published in 1965)
4. `SELECT title FROM books WHERE year >= 1986;` (Gives every book title published in or after 1986)
5. `SELECT title FROM books WHERE year BETWEEN 1965 AND 1986;` (Gives every book title published between 1965 and 1986, both years included)
6. `SELECT title FROM books WHERE year NOT BETWEEN 1986 AND 2018;` (Gives every book title published outside of 1986 to 2018, both years excluded)
7. `SELECT title, author FROM books WHERE year IN (1965, 1986);` (Gives book titles with authors, published in 1965 or 1986)
8. `SELECT author FROM books WHERE year NOT IN (2018);` (Gives every author except the one whose book came out in 2018)
9. `SELECT title FROM books WHERE author = 'MILLER';` (Returns the book title written by Miller)
10. `SELECT title FROM books WHERE author LIKE 'King';` (Returns the book title written by King)
11. `SELECT title FROM books WHERE author NOT LIKE 'Miller';` (Returns book titles not written by Miller)
12. `SELECT author FROM books WHERE author LIKE '%ING%';` (Returns the author named KING)
13. `SELECT author FROM books WHERE author LIKE 'MILLE_';` (Returns the author named MILLER)
14. `SELECT author FROM books WHERE title IN ('DUNE');` (Returns the author of DUNE)
15. `SELECT author FROM books WHERE title NOT IN ('IT');` (Returns every author except the one who wrote IT)
16. `SELECT DISTINCT title FROM books;` (Returns titles with duplicates removed)
17. `SELECT title, year FROM books ORDER BY year ASC;` (Returns every title with its year, oldest first)
18. `SELECT title, year FROM books ORDER BY year DESC LIMIT 3 OFFSET 1;` (Returns three books starting from the second most recent)

DATE: 23/06/2026

NORMALIZATION - Normalization cuts down on duplicate and redundant data inside a single table, letting data grow independently by spreading it across several tables. This means queries get more involved, since data now has to be pulled from different parts of the database, and performance can suffer once many large tables are in play. To answer a question about something whose data is spread across multiple normalized tables, a query needs to combine that data and extract exactly what is needed.

PRIMARY KEY - Any table that holds information about a single entity needs a PRIMARY KEY that identifies that entity uniquely across the whole database. An auto-incrementing integer is a common choice for a primary key because it is compact, but it could just as easily be a hash, a string, or anything else as long as it stays unique.

JOINS - The JOIN clause lets a query combine row data from two tables using their shared primary key.

INNER JOIN

An INNER JOIN matches rows from the first table with rows from the second table that share the same key, as defined by the ON condition, and produces a result row combining columns from both. Any remaining clauses are applied after the join happens.

NOTE - An INNER JOIN is sometimes written simply as JOIN — the two mean exactly the same thing.

```
SELECT column, another_table_column FROM my_table INNER JOIN another_table
ON my_table.id = another_table.id WHERE conditions ORDER BY column ASC/DESC
LIMIT num OFFSET num;
```

BOOKS

S_NO	TITLE	AUTHOR	YEAR
1	DUNE	HERBERT	1965
2	EMMA	AUSTEN	1815
3	IT	KING	1986
4	CIRCE	MILLER	2018

SALES

R_NO	RATING	COPIES SOLD	REVENUE
1	9	5,000	60,000
2	8	3,000	27,000
3	10	4,200	50,000
4	7	2,100	18,000

```
SELECT * FROM books INNER JOIN sales ON books.s_no = sales.r_no WHERE
rating > 7 ORDER BY year ASC LIMIT 2 OFFSET 1; (Gives full details for the
books tied to 1965 and 1986, ordered by year, restricted to a rating above
7)
```

OUTER JOINS - An INNER JOIN alone is not always enough, since it only keeps data that appears in both tables. When two tables hold asymmetric data — which happens easily when data is entered at different times — a LEFT JOIN, RIGHT JOIN, or FULL JOIN is needed instead, so nothing required gets left out of the results. Just like an INNER JOIN, each of these also needs a column to join on.

LEFT JOIN - Keeps every row from table A, whether or not a matching row exists in table B.

RIGHT JOIN - Keeps every row from table B, whether or not a matching row exists in table A.

FULL JOIN - Keeps every row from both tables, whether or not a matching row exists in the other one.

NOTE - With any of these three joins, columns from the unmatched side will come back as NULL, so extra handling with IS NULL or IS NOT NULL is usually needed.

NOTE - These joins are sometimes written as LEFT OUTER JOIN, RIGHT OUTER JOIN, or FULL OUTER JOIN — the OUTER keyword is kept only for SQL-92 compatibility, and behaves identically to LEFT JOIN, RIGHT JOIN, and FULL JOIN.

```
SELECT column, another_column FROM my_table LEFT/RIGHT/FULL JOIN
another_table ON my_table.id = another_table.matching_id WHERE conditions
ORDER BY column ASC/DESC LIMIT num OFFSET num;
```

BOOKS

S_NO	TITLE	AUTHOR	YEAR
1	DUNE	HERBERT	1965
2	EMMA	AUSTEN	1815
3	IT	KING	1986
4	CIRCE	MILLER	2018
5	SAPIENS	HARARI	2011

SALES

R_NO	RATING	COPIES SOLD	REVENUE
1	9	5,000	60,000
2	8	3,000	27,000
3	10	4,200	50,000
4	7	2,100	18,000
6	9	9,000	95,000

SELECT * FROM books LEFT JOIN sales ON books.s_no = sales.r_no; (Gives every book from the books table, including any that have no matching row in sales)

SELECT * FROM books RIGHT JOIN sales ON books.s_no = sales.r_no; (Gives every row from the sales table, including any that have no matching row in books)

SELECT * FROM books FULL JOIN sales ON books.s_no = sales.r_no; (Gives every row from both tables, including any with no counterpart in the other)

NULL - It helps to keep NULL values out of a database where possible, since they need special handling in queries, constraints, and functions (which can behave differently around NULLs), and when results are processed afterward. A data-type-appropriate default — 0 for numbers, an empty string for text — is often a workable alternative, but if the database genuinely needs to hold incomplete data, NULL is the right choice whenever a default would distort later analysis, such as when averaging numeric values.

NULLs are sometimes unavoidable, as seen already with outer-joining two tables holding asymmetric data. A column can be tested for NULL in a WHERE clause with IS NULL or IS NOT NULL.

```
SELECT column, another_column FROM my_table WHERE column IS/IS NOT NULL
AND/OR another_condition;
```

BOOKS

S_NO	TITLE	AUTHOR	YEAR
1	DUNE	HERBERT	1965

S_NO	TITLE	AUTHOR	YEAR
2	EMMA	AUSTEN	1815
3	IT	KING	1986
4	CIRCE	MILLER	2018
5	SAPIENS	NULL	2011

```
SELECT title FROM books WHERE author IS NULL; (Gives SAPIENS as the result)
SELECT title FROM books WHERE author IS NOT NULL; (Gives every title except SAPIENS)
```

QUERIES WITH EXPRESSIONS - Besides referencing raw column data, SQL also allows expressions that apply more complex logic to column values. These can mix mathematical and string functions with plain arithmetic to transform values as the query runs. Each database has its own set of supported math, string, and date functions, documented separately for each one. Expressions save time and cut down on post-processing, but can make a query harder to read — so whenever an expression appears in the SELECT part of a query, it is best given a descriptive alias with AS. Regular columns and even whole tables can be aliased too, to make complex queries easier to reference.

```
SELECT column_expression AS expression_description FROM my_table;
SELECT column_name AS better_column_name FROM long_table_name AS
better_table_name INNER JOIN another_table_name ON better_table_name.id =
another_table_name.id;
```

PRODUCTS

S_NO	PRODUCT	PRICE	QTY
1	Notebook	40	50
2	Pen	10	100

```
SELECT price / 2.0 AS discounted_price FROM products;
SELECT product AS products_available FROM products AS products_for_sale;
```

NOTE - *SELECT 25/2.0 returns 12.5 (decimal division, keeps the remainder). SELECT 25/2 returns 12 (integer division, drops the remainder). In SQL, dividing an integer by an integer gives an integer result — the decimal part disappears silently with no error. Adding .0 forces decimal division.*

AGGREGATE FUNCTIONS - On top of simple expressions, SQL supports aggregate expressions that summarize information across a group of rows. Without an explicit grouping, an aggregate function runs across the entire result set and returns one value. As with normal expressions, giving aggregate functions an alias makes the output easier to read and process.

```
SELECT AGG_FUNC (column_or_expression) AS aggregate_description FROM
my_table WHERE constraint_expression;
```

COMMON AGGREGATE FUNCTIONS

Function	Meaning
COUNT(*)	Counts the number of rows in the group when no column is named.
COUNT(column)	Counts the rows in the group where the named column is not NULL.
MIN(column)	Finds the smallest numeric value in the named column across the group.

Function	Meaning
MAX(column)	Finds the largest numeric value in the named column across the group.
AVG(column)	Finds the average numeric value in the named column across the group.
SUM(column)	Adds up all numeric values in the named column across the group.

BOOKS

S_NO	TITLE	AUTHOR	YEAR	PRICE
1	DUNE	HERBERT	1965	450
2	EMMA	AUSTEN	1815	300
3	IT	KING	1986	650
4	CIRCE	MILLER	2018	400
5	SAPIENS	NULL	2011	500

SELECT COUNT(*) FROM books; (Gives the total number of rows in the table)

SELECT COUNT(author) FROM books; (Gives the total number of rows in the author column, excluding NULLs)

SELECT MIN(price) FROM books; (Gives the cheapest book's price)

SELECT MAX(price) FROM books; (Gives the priciest book's price)

SELECT AVG(price) FROM books; (Gives the average price across all books)

SELECT SUM(price) FROM books; (Gives the combined price of all books)

DATE: 24/06/2026

GROUPED AGGREGATE FUNCTIONS - Instead of aggregating across every row at once, an aggregate function can be applied to individual groups of rows, as defined by the GROUP BY clause — one result per unique group. GROUP BY works by clustering rows that share the same value in the given column.

```
SELECT AGG_FUNC (column_or_expression) AS aggregate_description FROM
my_table WHERE constraint_expression GROUP BY column;
```

BOOKS

S_NO	TITLE	AUTHOR	YEAR	PRICE
1	DUNE	HERBERT	1965	450
2	EMMA	AUSTEN	1965	300
3	IT	KING	1986	650

SELECT SUM(price) AS total_price, year FROM books GROUP BY year; (Gives the combined price for 1965 and 1986)

SELECT AVG(price) AS average_price, year FROM books GROUP BY year; (Gives the average price for 1965 and 1986)


```
SELECT year, COUNT(*) AS total_books FROM books GROUP BY year; (Gives the number of books published in each year)
```

```
SELECT year, SUM(price) AS total_price FROM books WHERE price>300 GROUP BY year; (Gives the total price per year, dropping individual books priced 300 or under before grouping)
```

HAVING - The HAVING clause is used alongside GROUP BY to filter grouped rows out of the result set. Its constraints are written the same way as WHERE constraints, only they apply to grouped rows rather than individual ones. This may not look particularly useful in small examples, but with millions of rows carrying different properties, being able to filter after grouping quickly becomes necessary to make sense of the data.

```
SELECT column, AGG_FUNC(column_expression) AS aggregate_result_alias FROM my_table WHERE condition GROUP BY column HAVING group_condition;
```

NOTE - If GROUP BY is not being used, a plain WHERE clause is all that is needed.

BOOKS

S_NO	TITLE	AUTHOR	YEAR	PRICE
1	DUNE	HERBERT	1965	450
2	EMMA	AUSTEN	1986	300
3	IT	KING	1986	650
4	CIRCE	MILLER	1965	400
5	SAPIENS	NULL	2011	500

```
SELECT year, SUM(price) AS total_price FROM books WHERE year = 1965 GROUP BY year HAVING total_price > 400; (Gives the total price for years whose combined price exceeds 400)
```

NOTE - WHERE filters individual rows before grouping happens. HAVING filters entire groups after grouping. Both clauses can appear in the same query.

ORDER OF EXECUTION OF A QUERY - Every query starts by locating the needed data in a database, and then narrows it down into something that can be processed and understood as fast as possible. Since each part of a query runs in sequence, knowing that order matters for understanding which results are available at which stage.

```
SELECT DISTINCT column, AGG_FUNC(column_or_expression) FROM my_table JOIN another_table ON my_table.column = another_table.column WHERE constraint_expression GROUP BY column HAVING constraint_expression ORDER BY column ASC/DESC LIMIT count OFFSET count;
```

FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT / OFFSET

STEP-BY-STEP BREAKDOWN

1. FROM & JOIN (Gather the data): The database locates the tables named in the query. If a JOIN is merging data, the tables are combined into one working dataset first.
2. WHERE (Filter rows): The database scans the raw dataset and discards any row that fails the condition – done early to cut down what needs processing.
3. GROUP BY (Categorize): Remaining rows are bucketed by shared values in a chosen column, such as grouping sales data by region.

4. **HAVING (Filter groups):** If grouping happened, this acts as a second filter, discarding entire groups that fail a summary condition, such as keeping only groups where `SUM(sales) > 1000`.
5. **SELECT (Pick and compute columns):** Only now does the database look at `SELECT`, picking the requested columns and computing any expressions or aliases.
6. **DISTINCT (Remove duplicates):** If `DISTINCT` was used, the database scans the final column list and drops exact duplicate rows.
7. **ORDER BY (Sort):** The final data is sorted ascending or descending as requested.
8. **LIMIT / OFFSET (Slice):** Finally, the database returns only the exact number of rows requested and discards the rest.

NOTE - Because `WHERE` runs before `SELECT`, a column alias can never be used inside a `WHERE` clause — a common trap.

```
SELECT book_title AS title FROM books WHERE title = 'Dune'; (Gives an error)
```

When the database evaluates `WHERE`, it has not looked at `SELECT` yet, so the alias 'title' does not exist at that point. The same alias can, however, be used inside `ORDER BY`, since sorting happens after selection.

BRANCHES

S_NO	CITY	PRICE	YEAR
1	PUNE	300	2026
2	CHENNAI	550	2025
3	KOLKATA	700	2026
4	PUNE	350	2026
5	KOLKATA	150	2026

```
SELECT city, SUM(price) AS total_spent FROM branches WHERE year = 2026
GROUP BY city HAVING SUM(price) > 500 ORDER BY total_spent DESC LIMIT 3;
```

Step 1: FROM BRANCHES — The database pulls the entire table to start working with it.

Step 2: WHERE YEAR = 2026 — Rows where the year is not 2026 are dropped. (S_NO 2, CHENNAI, is removed since it is from 2025.)

Step 3: GROUP BY CITY — The remaining rows are bucketed by city. (PUNE bucket: 300, 350. KOLKATA bucket: 700, 150.)

Step 4: HAVING SUM(PRICE) > 500 — Each bucket's total is checked against the condition. (PUNE: 300 + 350 = 650, passes. KOLKATA: 700 + 150 = 850, passes.)

Step 5: SELECT CITY, SUM(PRICE) AS TOTAL_SPENT — The requested columns are now built and the math finalized.

CITY	TOTAL_SPENT
PUNE	650
KOLKATA	850

Step 6: ORDER BY TOTAL_SPENT DESC — Rows are sorted by the rule given (850 is bigger than 650, so Kolkata moves to the top).

Step 7: LIMIT 3 — Three rows were requested, but only two remain, so both are returned.

Final result delivered:

CITY	TOTAL_SPENT
KOLKATA	850
PUNE	650

DATABASE SCHEMA - A table has already been described as a two-dimensional grid of rows and columns, where columns are properties and rows are instances of the entity being stored. In SQL, the database schema describes the structure of every table and the data types allowed in each column. In a Books table, for instance, the Year column must hold an integer and the Title column must hold a string. This fixed structure is what lets a database stay efficient and consistent even while storing millions or billions of rows.

INSERTING ROWS - Adding data to a database uses an INSERT statement, which names the target table, the columns being filled, and one or more rows of values. Generally, every inserted row should carry a value for every corresponding column, and multiple rows can be inserted at once by listing them one after another.

```
INSERT INTO my_table VALUES (value1, another_value1), (value2,
another_value2);
```

Where data is incomplete and the table's columns support default values, rows can be inserted with only the columns available by naming them explicitly. In that case, the number of values given has to match the number of columns named.

```
INSERT INTO my_table (column, another_column) VALUES (value1,
another_value1), (value2, another_value2);
```

NOTE - Mathematical and string expressions can be used for the values being inserted, which helps keep inserted data formatted consistently.

BRANCHES

S_NO	CITY	PRICE	YEAR
1	PUNE	300	2026
2	CHENNAI	550	2025
3	KOLKATA	700	2026

```
INSERT INTO branches VALUES (6, 'NAGPUR', 400, 2023);
```

```
INSERT INTO branches (s_no, city, price, year) VALUES (7, 'INDORE', 600,
2022);
```

UPDATING ROWS - Changing existing data uses an UPDATE statement, which needs the exact table, columns, and rows to be specified, and the new values must match the schema's data types. The statement applies given column/value pairs to every row matching the WHERE condition.

```
UPDATE my_table SET column = value, another_column = another_value WHERE
condition;
```

NOTE - It is easy to make mistakes with UPDATE — whether that means touching the wrong rows in a live database or forgetting the WHERE clause entirely, which applies the update to every row. A safe habit is to write the WHERE condition first, test it as a SELECT query to confirm the right rows are targeted, and only then add the column/value pairs.

BRANCHES

S_NO	CITY	PRICE	YEAR
1	PUNE	300	2026
2	CHENNAI	550	2025
3	KOLKATA	700	2026

```
UPDATE branches SET price = 500, year = 2024 WHERE city = 'PUNE';
```

DELETING ROWS - Removing data from a table uses a DELETE statement, naming the table and the rows to remove through a WHERE clause. Skipping the WHERE clause removes every row, which is a quick way to empty a table entirely, provided that is intentional.

```
DELETE FROM my_table WHERE condition;
```

NOTE - Running the WHERE condition as a SELECT query first is a good way to confirm the right rows are about to be removed. Without a backup or a test database, data can be lost permanently, so DELETE statements are worth reading twice before running once.

```
DELETE FROM branches WHERE year = 2026 AND price = 300;
```

CREATING TABLES - New entities and relationships are stored by creating a new table with the CREATE TABLE statement. The new table's structure comes from its schema, defining a set of columns, where each column has a name, a data type, an optional constraint, and an optional default value.

```
CREATE TABLE IF NOT EXISTS mytable ( column DataType TableConstraint  
DEFAULT default_value, another_column DataType TableConstraint DEFAULT  
default_value);
```

NOTE - If a table with the same name already exists, most SQL implementations throw an error — IF NOT EXISTS suppresses that error and skips creation instead.

TABLE DATA TYPES

Different databases support different data types, but common categories cover numbers, strings, and other items like dates, booleans, or binary data.

1. NUMBERS

INTEGER / BOOLEAN: Stores whole numbers such as 5, 42, or -10. Databases often use 0 for false and 1 for true.

FLOAT / DOUBLE / REAL: Stores numbers with decimals, such as 3.14 or 99.99, useful for values that need precision.

2. TEXT

CHARACTER (CHAR): Fixed-length text. Setting it to 5 characters and entering "hi" pads the rest with spaces — useful for fixed-size codes such as state abbreviations.

VARCHAR: Variable-length text, using only as much space as the word needs, up to a set limit.

TEXT: Meant for long content, such as an article or a review.

3. DATES AND TIME

DATE / DATETIME: Stores calendar dates and timestamps, such as 2026-06-24 19:02:00 — useful for tracking when things happen, though different time zones can complicate things.

4. EVERYTHING ELSE (FILES/BINARY)

BLOB (Binary Large Object): Stores raw data such as images, audio, or PDFs directly inside the database. The database does not "read" the file — it simply holds onto it like a storage locker.

NOTE - (i) FLOAT or DOUBLE should never be used for currency — rounding errors will eventually throw financial totals off by fractions of a unit. DECIMAL or NUMERIC exists specifically to store exact decimals for money.

NOTE - (ii) TEXT is not a good default for everything, since it slows queries down. A database prefers to know exactly how wide a row is in memory — CHAR and VARCHAR let it skip through memory quickly, while TEXT forces it to follow a pointer to another location to read the data.

NOTE - (iii) DATETIME is naive — it stores exactly what is entered and ignores time zones. TIMESTAMP converts the time to UTC before saving it, and converts it back to a user's local time zone when that data is requested.

NOTE - (iv) A BLOB is just a large block of ones and zeros — the database has no idea what is inside it. If an image is uploaded, the raw binary file is stored as-is, and backend code has to pull that binary stream and tell the browser how to treat it, such as as a .png file. Storing large files like videos or high-resolution photos inside SQL is generally discouraged today, since it makes backups slow and eats up costly database storage. Most developers store the actual file in a cloud storage bucket, such as Amazon S3, and save only a VARCHAR pointing to its URL in the database.

DATE: 25/06/2026

TABLE CONSTRAINTS - Each column can carry extra constraints that limit what can be inserted into it.

PRIMARY KEY - Values in this column are unique, and each one can identify a single row in the table.

AUTOINCREMENT - For integer columns, the value is filled in automatically and incremented with each new row. Not every database supports this.

UNIQUE - Values in this column must be unique, so no two rows can share the same value here — unlike PRIMARY KEY, it does not have to serve as the row's identifying key.

NOT NULL - The inserted value cannot be NULL.

CHECK (expression) - Runs a more complex expression to validate inserted values — for example, requiring values to be positive, above a certain size, or starting with a given prefix.

FOREIGN KEY - A consistency check ensuring that each value in this column corresponds to a value in a column of another table.

BRANCHES

S_NO	CITY	PRICE	YEAR
1	PUNE	300	2026
2	CHENNAI	550	2025
3	KOLKATA	700	2026

```
CREATE TABLE IF NOT EXISTS branches (s_no INTEGER PRIMARY KEY, city TEXT, price INTEGER, year INTEGER);
```

ALTERING TABLE - As data needs change over time, SQL provides the ALTER TABLE statement to add, remove, or modify columns and constraints in an existing table.

ADDING COLUMNS - Adding a column looks similar to defining one in CREATE TABLE — a data type, any constraints, and a default value applied to both existing and new rows. Some databases, such as MySQL, allow placing the new column with FIRST or AFTER, though this is not a standard feature.

```
ALTER TABLE my_table ADD column DataType OptionalTableConstraint DEFAULT default_value;
```

REMOVING COLUMNS - Dropping a column just means naming it, though some databases, including SQLite, do not support this directly — a new table and a data migration may be needed instead.

```
ALTER TABLE my_table DROP column_to_be_deleted;
```

RENAMING THE TABLE - A table itself can be renamed using RENAME TO.

```
ALTER TABLE my_table RENAME TO new_table_name;
```

BRANCHES

S_NO	CITY	PRICE	YEAR
1	PUNE	300	2026
2	CHENNAI	550	2025
3	KOLKATA	700	2026

```
ALTER TABLE branches ADD stock INTEGER DEFAULT 0;
```

```
ALTER TABLE branches DROP stock;
```

```
ALTER TABLE branches RENAME TO all_branches;
```

DROPPING TABLES - To remove an entire table, along with all its data and its schema, the DROP TABLE statement is used — different from DELETE, which only removes data and leaves the schema in place. As with CREATE TABLE, the database may throw an error if the table does not exist, which IF EXISTS suppresses. If another table depends on columns being removed, such as through a FOREIGN KEY, dependent rows or tables need to be handled first.

```
DROP TABLE IF EXISTS my_table;
```

BRANCHES

S_NO	CITY	PRICE	YEAR
1	PUNE	300	2026
2	CHENNAI	550	2025
3	KOLKATA	700	2026

```
DROP TABLE IF EXISTS branches;
```

POINTS TO REMEMBER

1. Every column in SELECT that is not wrapped in an aggregate function (SUM, COUNT, AVG, etc.) must also appear in GROUP BY.

```
SELECT city, staff_name, COUNT(*) FROM staff_directory GROUP BY city;
(Wrong)
```

This fails because once rows are grouped by city, a city can hold many staff members, and SQL has no way of knowing which name to display.

```
SELECT city, staff_name, COUNT(*) FROM staff_directory GROUP BY city,
staff_name; (Correct)
```

Now each row represents one staff member within one city, so the result is unambiguous.

2. If grouping by a column, that column should also appear in SELECT so the result stays readable.

QUESTION SET 1 - CONCEPT CHECK

STAFF

staff_id	name	dept	pay	join_year	lead_id
1	Anita	Sales	42000	2020	NULL
2	Rohan	Tech	75000	2019	NULL
3	Kavya	Tech	69000	2021	2
4	Farhan	Sales	39000	2022	1
5	Meena	Tech	57000	2021	2
6	Suresh	Finance	61000	2020	NULL
7	Leela	Finance	59000	2022	6
8	Vikram	Tech	NULL	2023	2

TASKS

task_id	staff_id	task_name	budget
201	2	Orion	480000
202	3	Nova	310000
203	5	Orion	480000
204	6	Comet	460000
205	9	Nebula	210000

1. Get the names and pay of every staff member in the Tech department.

```
SELECT name, pay FROM staff WHERE dept = 'Tech';
```

2. Find every staff member who joined between 2020 and 2022 (inclusive) and earns more than 45,000.

```
SELECT * FROM staff WHERE join_year BETWEEN 2020 AND 2022 AND pay > 45000;
```

3. Find every staff member whose name ends with the letter 'a'.

```
SELECT * FROM staff WHERE name LIKE '%a';
```

4. List every unique department in the company.

```
SELECT DISTINCT dept FROM staff;
```

5. Get the 2nd and 3rd highest-paid staff members (name and pay only).

```
SELECT name, pay FROM staff ORDER BY pay DESC LIMIT 2 OFFSET 1;
```

6. Find every staff member with no assigned lead.

```
SELECT * FROM staff WHERE lead_id IS NULL;
```

7. Show each staff member's name alongside their task name. Only include staff who have a task.

```
SELECT name, task_name FROM staff INNER JOIN tasks ON staff.staff_id = tasks.staff_id;
```

8. Show every staff member and their task name (if any). Staff without a task should still appear.

```
SELECT name, task_name FROM staff LEFT JOIN tasks ON staff.staff_id = tasks.staff_id;
```

9. Find the total pay bill, average pay, and the number of staff in the company. Exclude staff with a NULL pay.

```
SELECT SUM(pay) AS Total_Pay_Bill, AVG(pay) AS Average_Pay, COUNT(pay) AS Total_Staff FROM staff;
```

10. For each department, find the total pay. Show only departments where the total pay exceeds 1,00,000.

```
SELECT dept, SUM(pay) AS Total_Pay FROM staff GROUP BY dept HAVING SUM(pay) > 100000;
```

11. Show each staff member's name and their pay after an 8% raise. Label the new column raised_pay.

```
SELECT name, (pay*1.08) AS raised_pay FROM staff;
```

12. Add a new staff member: staff_id 9, name 'Divya', dept 'Sales', pay 44000, join_year 2024, lead_id 1.

```
INSERT INTO staff VALUES (9, 'Divya', 'Sales', 44000, 2024, 1);
```

13. Vikram (staff_id 8) has just been assigned a pay of 65000. Update his record.

```
UPDATE staff SET pay = 65000 WHERE staff_id = 8;
```

14. Remove every staff member who joined in 2023.

```
DELETE FROM staff WHERE join_year = 2023;
```

15. Create a table called departments with columns: dept_id (integer, primary key), dept_name (text, not null), and location (text, default 'Remote').

```
CREATE TABLE IF NOT EXISTS departments (dept_id INTEGER PRIMARY KEY, dept_name TEXT NOT NULL, location TEXT DEFAULT 'Remote');
```

16. Add a column contact VARCHAR(15) to the staff table.

```
ALTER TABLE staff ADD contact VARCHAR(15);
```

17. Drop the departments table safely (without throwing an error if it does not exist).

```
DROP TABLE IF EXISTS departments;
```

QUESTION SET 2 - REAL-WORLD ANALYST SCENARIO

You are a data analyst at BookNest, an online bookstore. The BI team has asked you to pull insights from the database.

MEMBERS

member_id	name	city	joined_year
1	Neha Kapoor	Pune	2021
2	Aman Verma	Delhi	2020
3	Ritu Sharma	Bengaluru	2022

member_id	name	city	joined_year
4	Vikas Joshi	Pune	2020
5	Sonal Iyer	Chennai	2023
6	Karan Malhotra	Delhi	2021

PURCHASES

purchase_id	member_id	item	category	amount	purchase_year	status
5001	1	Novel	Fiction	500	2023	Delivered
5002	2	Notebook	Stationery	150	2023	Delivered
5003	1	Tablet	Electronics	22000	2024	Delivered
5004	3	Earphones	Electronics	1800	2023	Cancelled
5005	4	Sneakers	Footwear	2200	2024	Delivered
5006	2	Tablet	Electronics	22000	2024	Delivered
5007	6	Watch	Accessories	4500	2023	Delivered
5008	5	Backpack	Accessories	1200	2024	Delivered
5009	3	Novel	Fiction	500	2024	Delivered
5010	NULL	Speaker	Electronics	3000	2024	Delivered

1. The marketing team wants to know how many members are from each city. Sort by count, highest first.

```
SELECT COUNT(city) AS City_Count FROM members GROUP BY city ORDER BY
COUNT(city) DESC;
```

```
SELECT city, COUNT(*) AS City_Count FROM members GROUP BY city ORDER BY
City_Count DESC; (Correct approach)
```

2. The finance team needs total revenue per item category, but only for delivered orders. Exclude categories with total revenue under Rs. 5,000.

```
SELECT SUM(amount) AS Total_Revenue FROM purchases WHERE status =
'Delivered' GROUP BY category HAVING SUM(amount) >= 5000;
```

```
SELECT category, SUM(amount) AS Total_Revenue FROM purchases WHERE status =
'Delivered' GROUP BY category HAVING SUM(amount) > 5000; (Correct approach)
```

3. Find members who have spent more than Rs. 20,000 in total across their delivered purchases. Show their name, city, and total spend.

```
SELECT name, city, SUM(amount) AS Total_Spend FROM members INNER JOIN
purchases ON members.member_id = purchases.member_id WHERE status =
'Delivered' GROUP BY member_id HAVING SUM(amount) > 20000;
```

```
SELECT name, city, SUM(amount) AS Total_Spend FROM members INNER JOIN
purchases ON members.member_id = purchases.member_id WHERE status =
'Delivered' GROUP BY members.member_id, name, city HAVING SUM(amount) >
20000; (Correct approach)
```

4. There is a purchase with no linked member. Write a query to find all purchases with no matching member record.

```
SELECT * FROM purchases WHERE member_id IS NULL;
```

5. The CXO wants the single highest purchase amount placed from each city. Show the city and the max amount.

```
SELECT city, MAX(amount) AS Highest_Amount FROM purchases INNER JOIN  
members ON purchases.member_id = members.member_id GROUP BY city;
```

6. Show each category's total revenue and what percentage it contributes to overall revenue. Label it revenue_pct. Round to 1 decimal place using ROUND(value, 1).

```
SELECT category, SUM(amount) AS Total_Revenue GROUP BY category;  
  
SELECT category, SUM(amount) AS Total_Revenue,  
ROUND(SUM(amount)*100.0/(SELECT SUM(amount) FROM purchases), 1) AS  
revenue_pct FROM purchases GROUP BY category; (Correct approach)
```

7. A GDPR-style request came in: delete all purchases placed by member Sonal Iyer. Write the safest version of this query.

```
DELETE FROM purchases WHERE member_id = (SELECT member_id FROM members  
WHERE name = 'Sonal Iyer');
```

8. The product team wants to track dispatch dates. Add a column dispatch_date DATE to the purchases table.

```
ALTER TABLE purchases ADD dispatch_date DATE;
```